


INSTITUTO TECNOLÓGICO DE SAN LUIS POTOSÍ

# Laboratorio de captura y análisis de tramas de red

## Tópicos de seguridad informática

**NOMBRE DE LOS ALUMNOS:**
Hernández Mendoza José Aurelio
**CARRERA:** Ingeniería en sistemas computacionales

AHUALULCO, S.L.P., A 05 DE AGOSTO DEL 2026

# Contenido

Introducción ..... 1

Práctica 1. Captura de tráfico con Wireshark ..... 2

Práctica 2. Captura de tráfico con Microsoft Pktmon ..... 4

Práctica 3. Captura de tráfico con tcpdump ..... 8

Practica 4. transferir paquetes de Ubuntu Server a Windows ..... 11

Practica 5. Analizar el Three-Way Handshake en Wireshark (Windows 11) ..... 13

Tabla Comparativa de Herramientas de Captura de Tráfico ..... 15

Preguntas de reflexión ..... 17

Conclusión ..... 18

**Introducción**

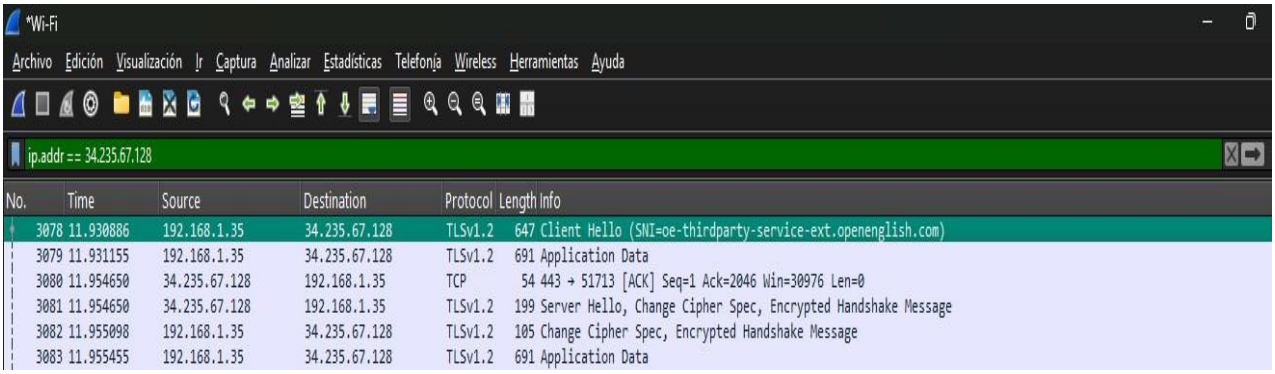
En esta actividad presentaremos un laboratorio de análisis de tramas de red, por medio de la interfaz grafica de wireshark que no permite ver como se comunican los protocolos en la red, gracias en su interfaz donde podemos filtrar entre protocolos y para guardar evidencia, y volver a analizarla en wireshark, usando también Microsoft Pktmon en línea de comandos para capturar y guardar paquetes y convertirlos en formato PCAPNG para poder analizarlo después en wireshark y por ultimo tcpdump que es la versión de Linux para capturar trafico de paquetes y guardar la evidencia encontrada.

**Práctica 1. Captura de tráfico con Wireshark**

**Captura de tráfico HTTPS (TLS 1.2) hacia servicio web de open english**

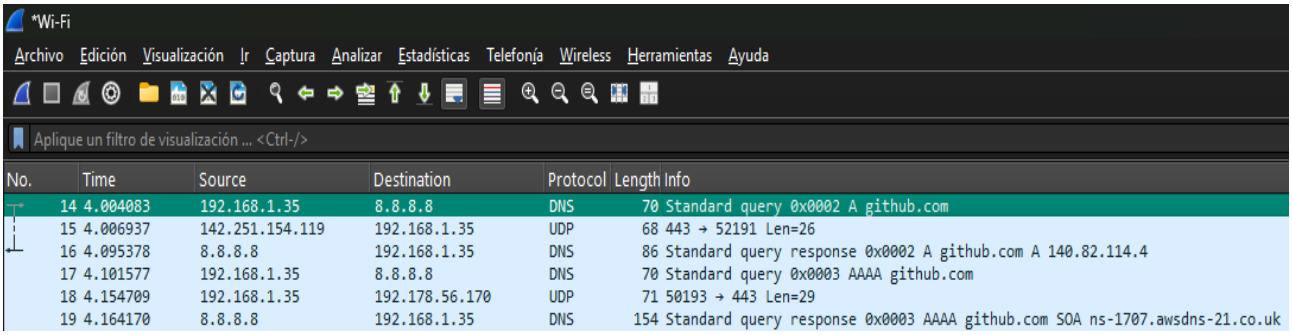
En la siguiente captura podemos ver cómo se comunican los paquetes donde client hello que viene siendo mi ip local 192.168.1.35 le manda la petición al servidor con ip 34.235.67.128 y el servidor responde server hello, para después hacer la conexión como es protocolo https la información aparece cifrada como Aplication Data

Esto viene siendo como nuestra red se conecta al servidor de open english para nosotros poder iniciar sesión y ver nuestras clases y practicas



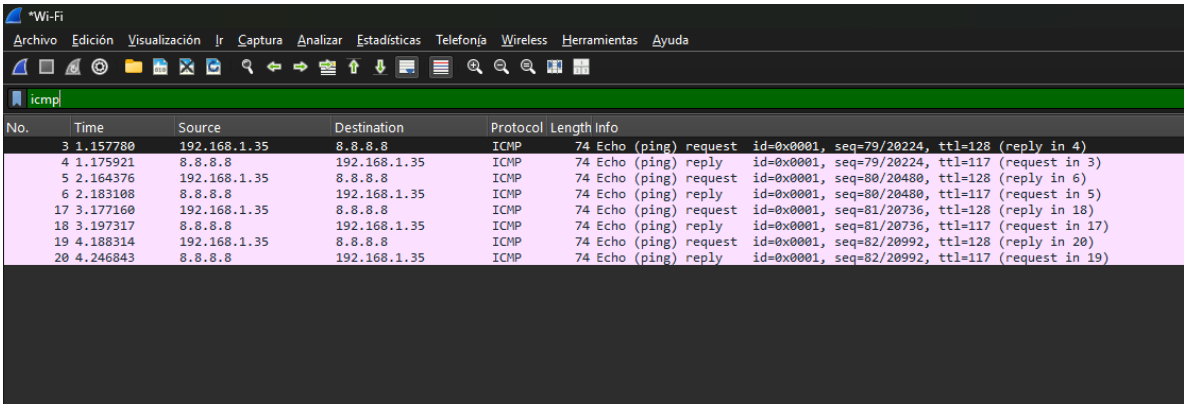

**Realizar una consulta DNS**

Aquí en esta captura podemos ver donde el primer paqute esta un query y hace la consulta del dominio GitHub.com, para después ver la respuesta query response donde entrega la ip en formato ipv4. Aquí aprendemos como se comunican los paquetes para poder ver la ip de un dominio o viceversa por medio del protocolo DNS



**Ejecutar un comando ping hacia un servidor público**

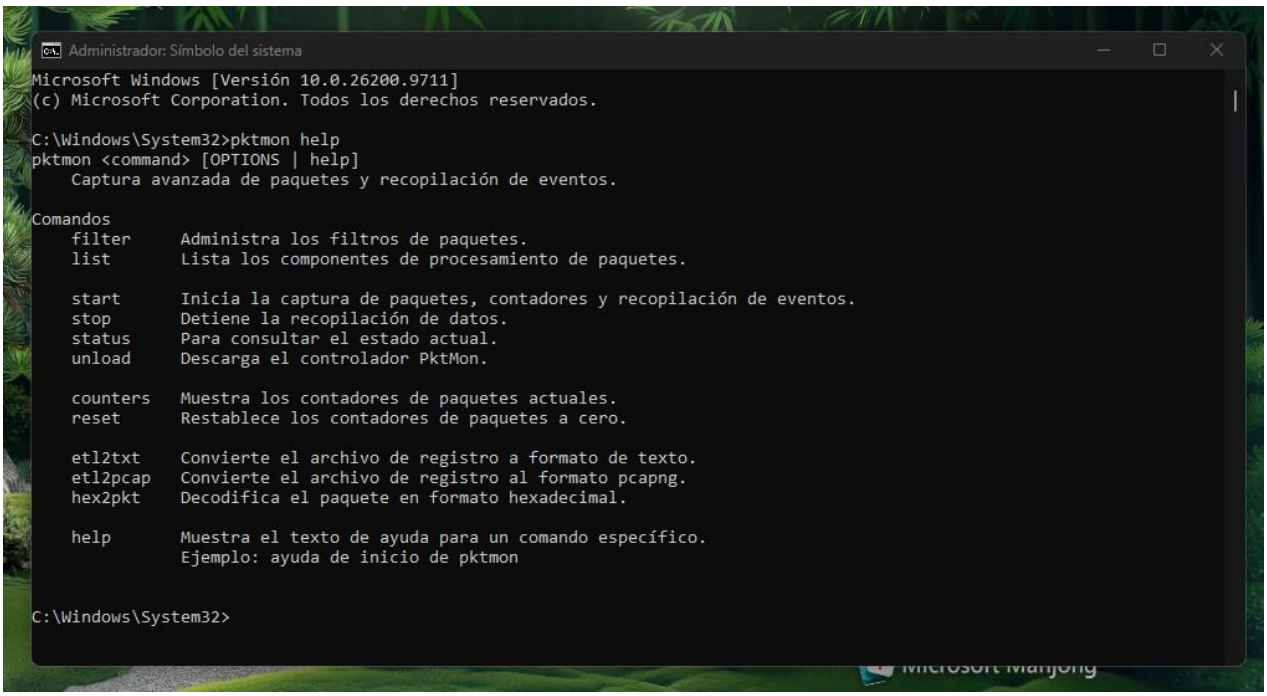
En esta captura se donde usamos el filtrado de ICMP porque es el protocolo que se encarga de diagnostico por medio de comandos como ping, en este caso un ping a 8.8.8.8 que es el servicio público de Google, bueno vemos que como mi ip local 192.168.1.35 hace la petición y la ip 8.8.8.8 del servidor responde al ping. Aquí podemos aprender como con el protocolo ICMP podemos hacer diagnósticos de conexión ósea para verificar si mi red si se está conectado con el servidor por medio de un ping



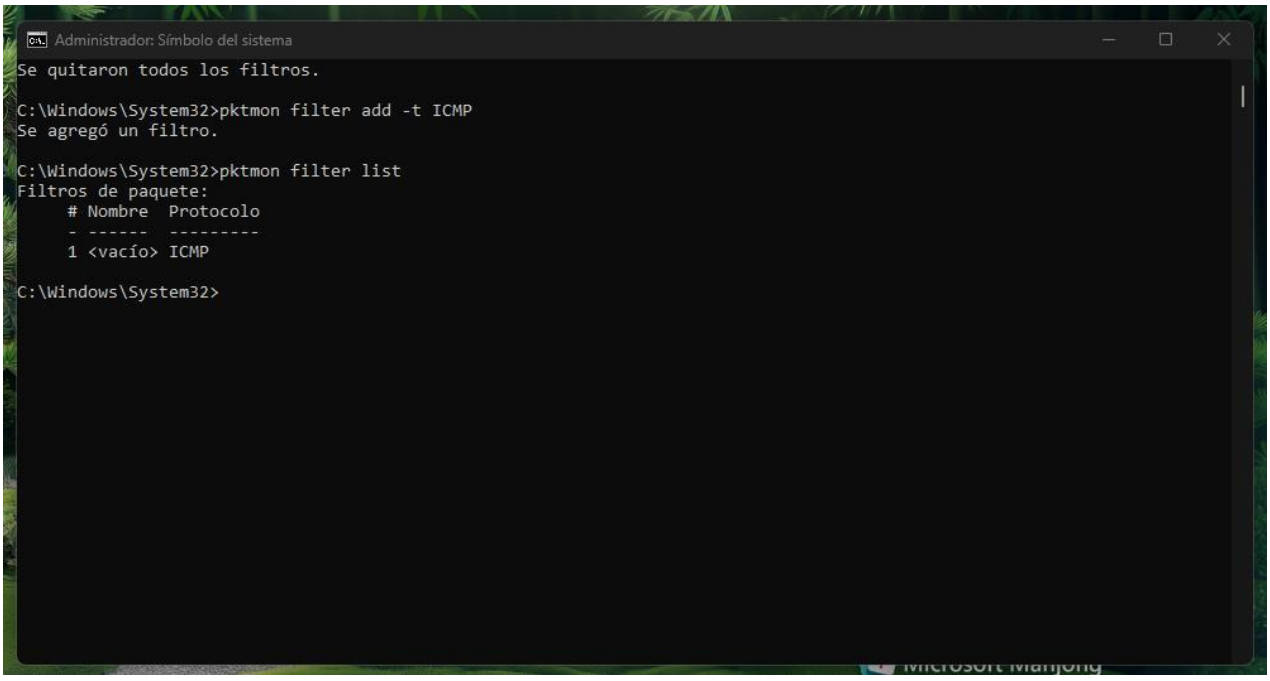

En esta práctica identificamos varios protocolos como lo es ICMP para diagnostico de dispositivos, TCP que es el se encarga del transporte de los datos por medio de peticiones, DNS que es el que traduce ips a nombres de dominios, DHCP es el que otorga ips y por último HTTPS que es el que protege los datos mientras viajan en internet por medio de cifrado.

**Práctica 2. Captura de tráfico con Microsoft Pktmon**

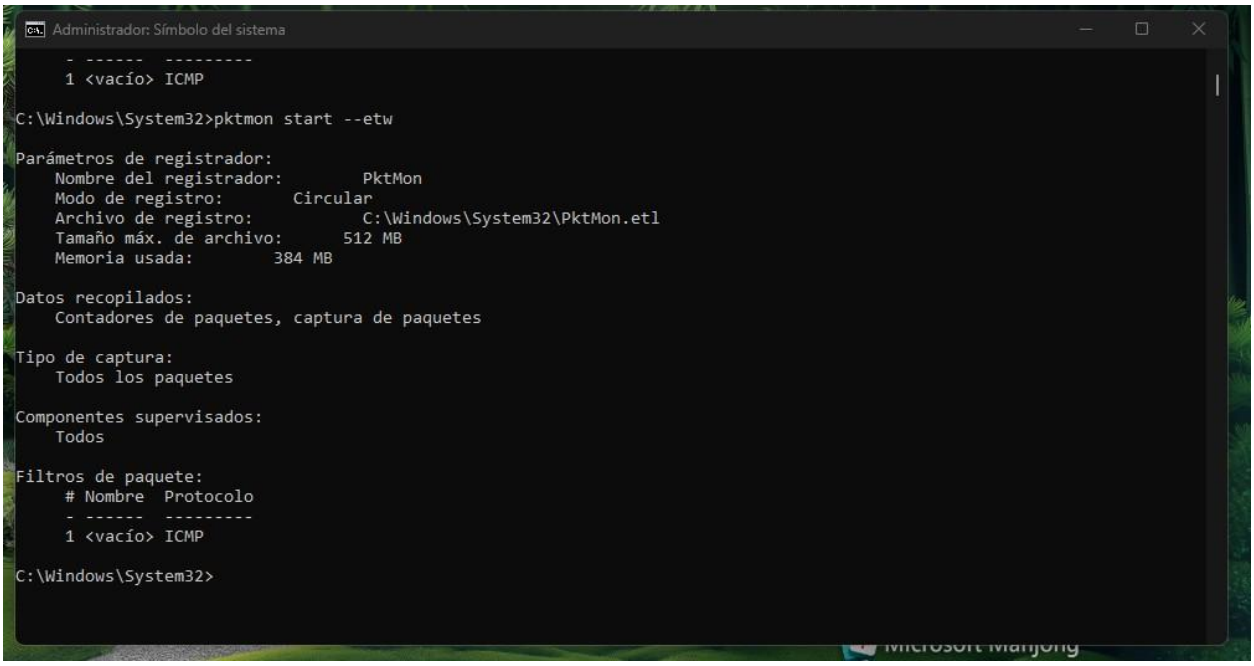
Primero entramos a la terminal como administrador y consultamos las funciones disponibles con el comando **pktmon help** como se muestra en la siguiente captura



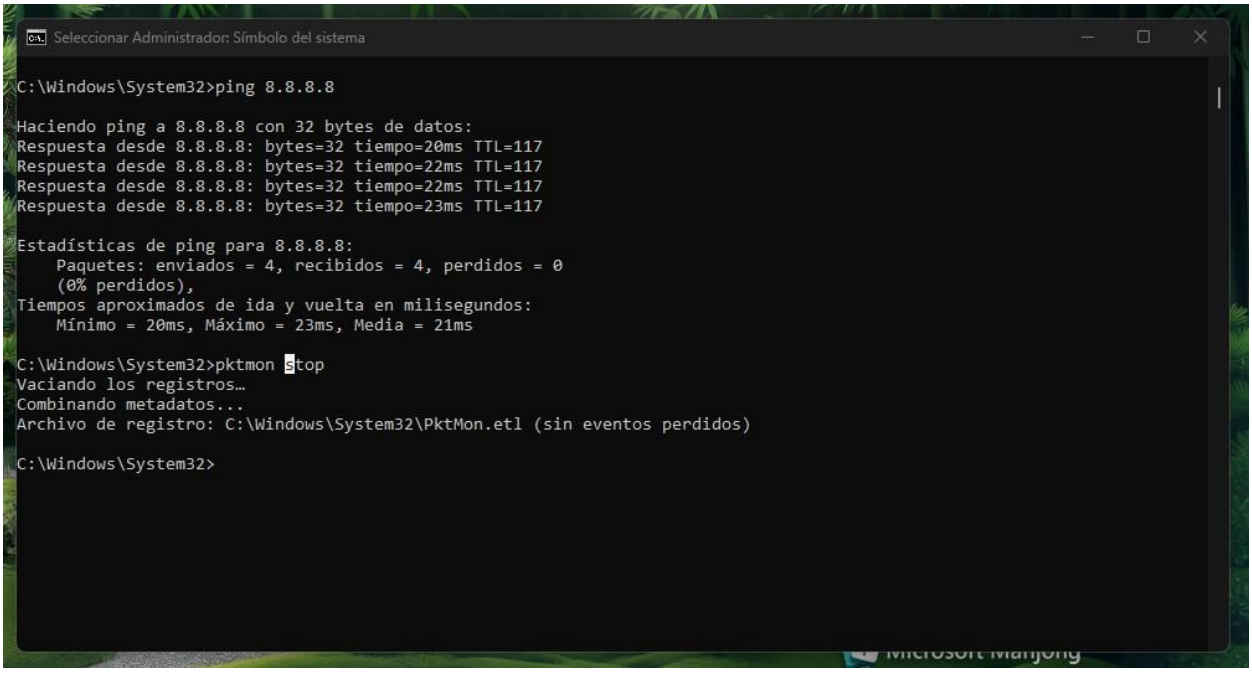

Lo siguiente es iniciar la captura de paquetes con el comando **start** pero antes para evitar capturar todo el tráfico que no vamos ocupare, añadí el filtro **pktmon filter add -t ICMP** para ICMP y verificamos que quede registrado con el comando **pktmon filter list**



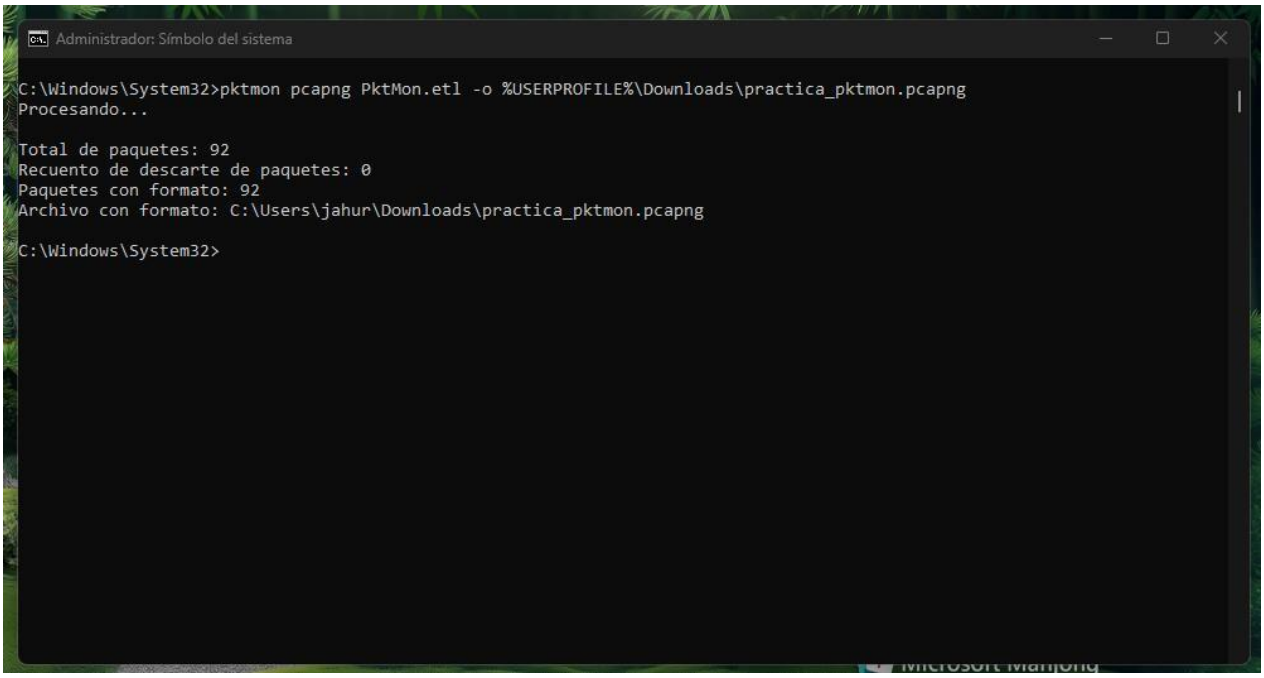
Ahora iniciamos la captura de paquetes con el comando **pktmon start –etw**



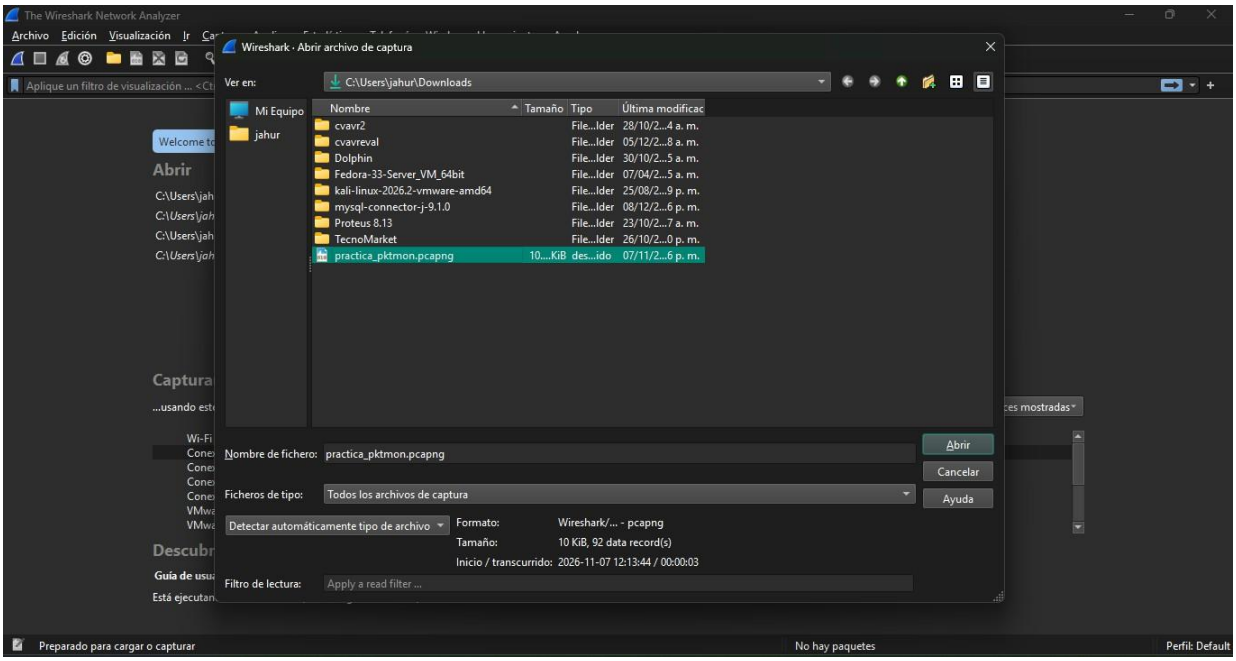

Ahora que ya iniciamos la captura vamos a crear trafico con un ping a 8.8.8.8 y luego detenemos la captura con el comando **pktmon stop** y vemos que se guardo la captura



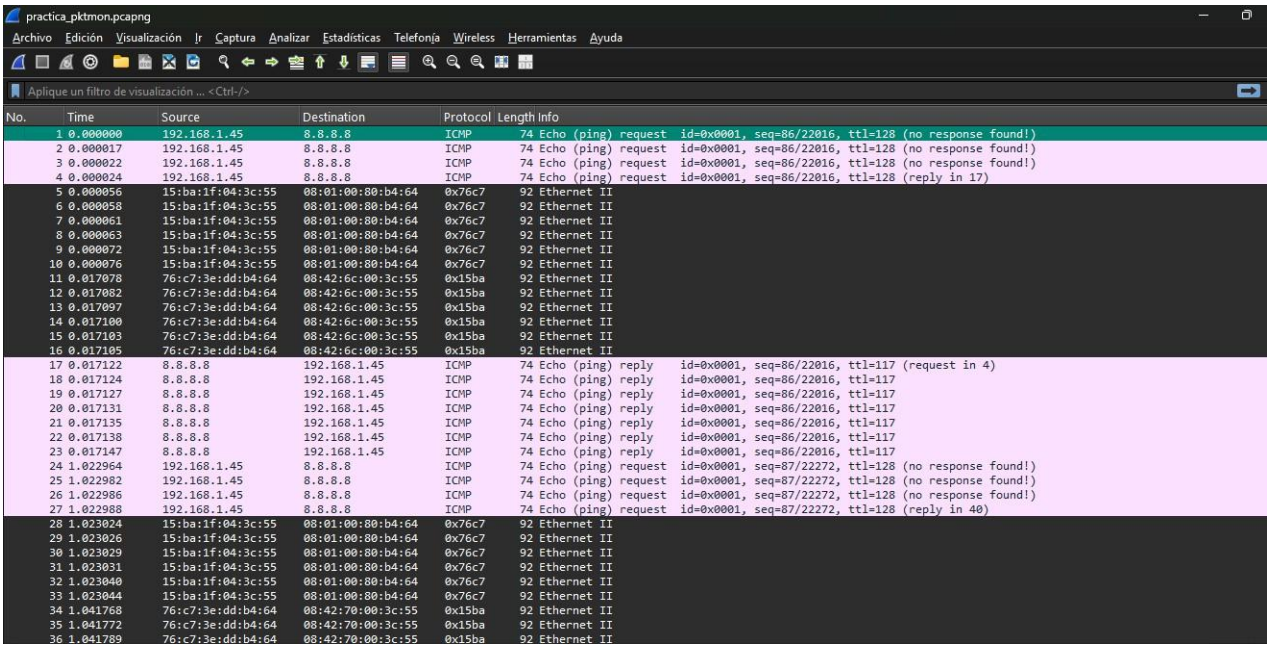
Convertir la captura guardada a formato PCAPNG para Wireshark con el comando **pktmon pcapng PktMon.etl -o %USERPROFILE%\Downloads\practica_pktmon.pcapng**




Después entramos a wireshark para abrir el archivo que se guardo en descargas y lo abrimos



En la siguiente imagen vemos que se importó correctamente el archivo en wireshark



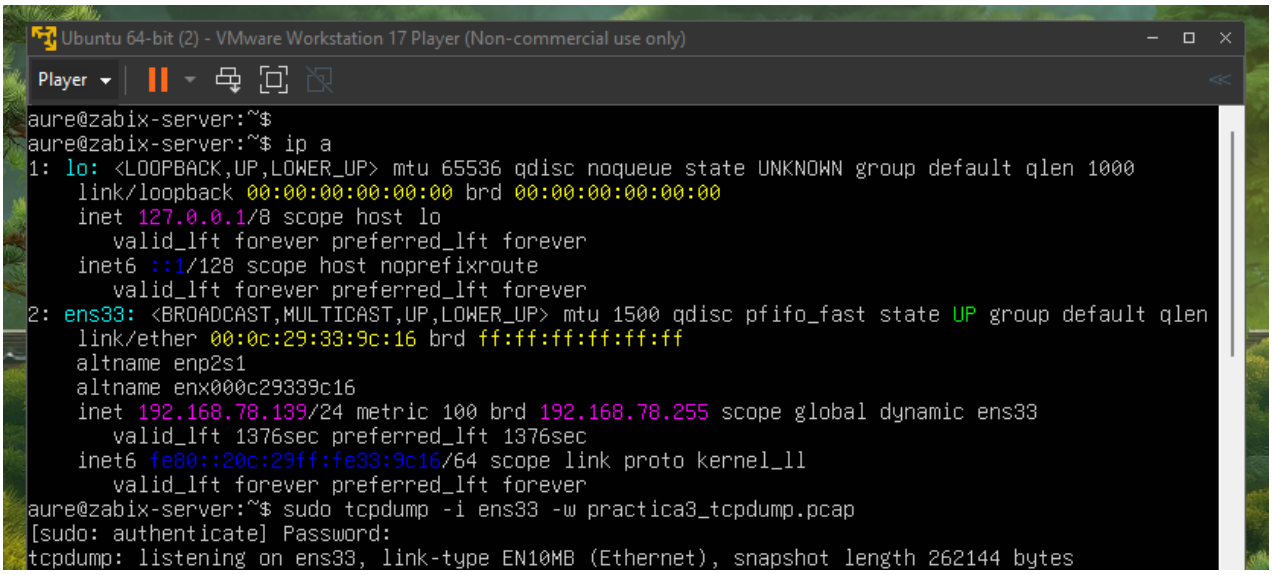

| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 1 | 0.000000 | 192.168.1.45 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x0001, seq=86/22016, ttl=128 (no response found!) |
| 2 | 0.000017 | 192.168.1.45 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x0001, seq=86/22016, ttl=128 (no response found!) |
| 3 | 0.000022 | 192.168.1.45 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x0001, seq=86/22016, ttl=128 (no response found!) |
| 4 | 0.000024 | 192.168.1.45 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x0001, seq=86/22016, ttl=128 (reply in 17) |
| 5 | 0.000056 | 15:ba:1f:04:3c:55 | 08:01:00:80:b4:64 | 0x76c7 | 92 | Ethernet II |
| 6 | 0.000058 | 15:ba:1f:04:3c:55 | 08:01:00:80:b4:64 | 0x76c7 | 92 | Ethernet II |
| 7 | 0.000061 | 15:ba:1f:04:3c:55 | 08:01:00:80:b4:64 | 0x76c7 | 92 | Ethernet II |
| 8 | 0.000063 | 15:ba:1f:04:3c:55 | 08:01:00:80:b4:64 | 0x76c7 | 92 | Ethernet II |
| 9 | 0.000072 | 15:ba:1f:04:3c:55 | 08:01:00:80:b4:64 | 0x76c7 | 92 | Ethernet II |
| 10 | 0.000076 | 15:ba:1f:04:3c:55 | 08:01:00:80:b4:64 | 0x76c7 | 92 | Ethernet II |
| 11 | 0.017078 | 76:c7:3e:dd:b4:64 | 08:42:6c:00:3c:55 | 0x15ba | 92 | Ethernet II |
| 12 | 0.017082 | 76:c7:3e:dd:b4:64 | 08:42:6c:00:3c:55 | 0x15ba | 92 | Ethernet II |
| 13 | 0.017097 | 76:c7:3e:dd:b4:64 | 08:42:6c:00:3c:55 | 0x15ba | 92 | Ethernet II |
| 14 | 0.017100 | 76:c7:3e:dd:b4:64 | 08:42:6c:00:3c:55 | 0x15ba | 92 | Ethernet II |
| 15 | 0.017103 | 76:c7:3e:dd:b4:64 | 08:42:6c:00:3c:55 | 0x15ba | 92 | Ethernet II |
| 16 | 0.017105 | 76:c7:3e:dd:b4:64 | 08:42:6c:00:3c:55 | 0x15ba | 92 | Ethernet II |
| 17 | 0.017122 | 8.8.8.8 | 192.168.1.45 | ICMP | 74 | Echo (ping) reply id=0x0001, seq=86/22016, ttl=117 (request in 4) |
| 18 | 0.017124 | 8.8.8.8 | 192.168.1.45 | ICMP | 74 | Echo (ping) reply id=0x0001, seq=86/22016, ttl=117 |
| 19 | 0.017127 | 8.8.8.8 | 192.168.1.45 | ICMP | 74 | Echo (ping) reply id=0x0001, seq=86/22016, ttl=117 |
| 20 | 0.017131 | 8.8.8.8 | 192.168.1.45 | ICMP | 74 | Echo (ping) reply id=0x0001, seq=86/22016, ttl=117 |
| 21 | 0.017135 | 8.8.8.8 | 192.168.1.45 | ICMP | 74 | Echo (ping) reply id=0x0001, seq=86/22016, ttl=117 |
| 22 | 0.017138 | 8.8.8.8 | 192.168.1.45 | ICMP | 74 | Echo (ping) reply id=0x0001, seq=86/22016, ttl=117 |
| 23 | 0.017147 | 8.8.8.8 | 192.168.1.45 | ICMP | 74 | Echo (ping) reply id=0x0001, seq=86/22016, ttl=117 |
| 24 | 1.022964 | 192.168.1.45 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x0001, seq=87/22272, ttl=128 (no response found!) |
| 25 | 1.022982 | 192.168.1.45 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x0001, seq=87/22272, ttl=128 (no response found!) |
| 26 | 1.022986 | 192.168.1.45 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x0001, seq=87/22272, ttl=128 (no response found!) |
| 27 | 1.022988 | 192.168.1.45 | 8.8.8.8 | ICMP | 74 | Echo (ping) request id=0x0001, seq=87/22272, ttl=128 (reply in 40) |
| 28 | 1.023024 | 15:ba:1f:04:3c:55 | 08:01:00:80:b4:64 | 0x76c7 | 92 | Ethernet II |
| 29 | 1.023026 | 15:ba:1f:04:3c:55 | 08:01:00:80:b4:64 | 0x76c7 | 92 | Ethernet II |
| 30 | 1.023029 | 15:ba:1f:04:3c:55 | 08:01:00:80:b4:64 | 0x76c7 | 92 | Ethernet II |
| 31 | 1.023031 | 15:ba:1f:04:3c:55 | 08:01:00:80:b4:64 | 0x76c7 | 92 | Ethernet II |
| 32 | 1.023040 | 15:ba:1f:04:3c:55 | 08:01:00:80:b4:64 | 0x76c7 | 92 | Ethernet II |
| 33 | 1.023044 | 15:ba:1f:04:3c:55 | 08:01:00:80:b4:64 | 0x76c7 | 92 | Ethernet II |
| 34 | 1.041768 | 76:c7:3e:dd:b4:64 | 08:42:70:00:3c:55 | 0x15ba | 92 | Ethernet II |
| 35 | 1.041772 | 76:c7:3e:dd:b4:64 | 08:42:70:00:3c:55 | 0x15ba | 92 | Ethernet II |
| 36 | 1.041789 | 76:c7:3e:dd:b4:64 | 08:42:70:00:3c:55 | 0x15ba | 92 | Ethernet II |

En la inspección del tráfico se observa el registro de los eventos ICMP generados durante la prueba de conectividad con ping 8.8.8.8. Debido al funcionamiento de Microsoft Pktmon mediante Event Tracing for Windows, se registran múltiples eventos por cada paquete a medida que atraviesa las distintas capas del stack de red de Windows, confirmándose la salida de las solicitudes de eco y la llegada de las respuestas correspondientes enviadas por el host público 8.8.8.8

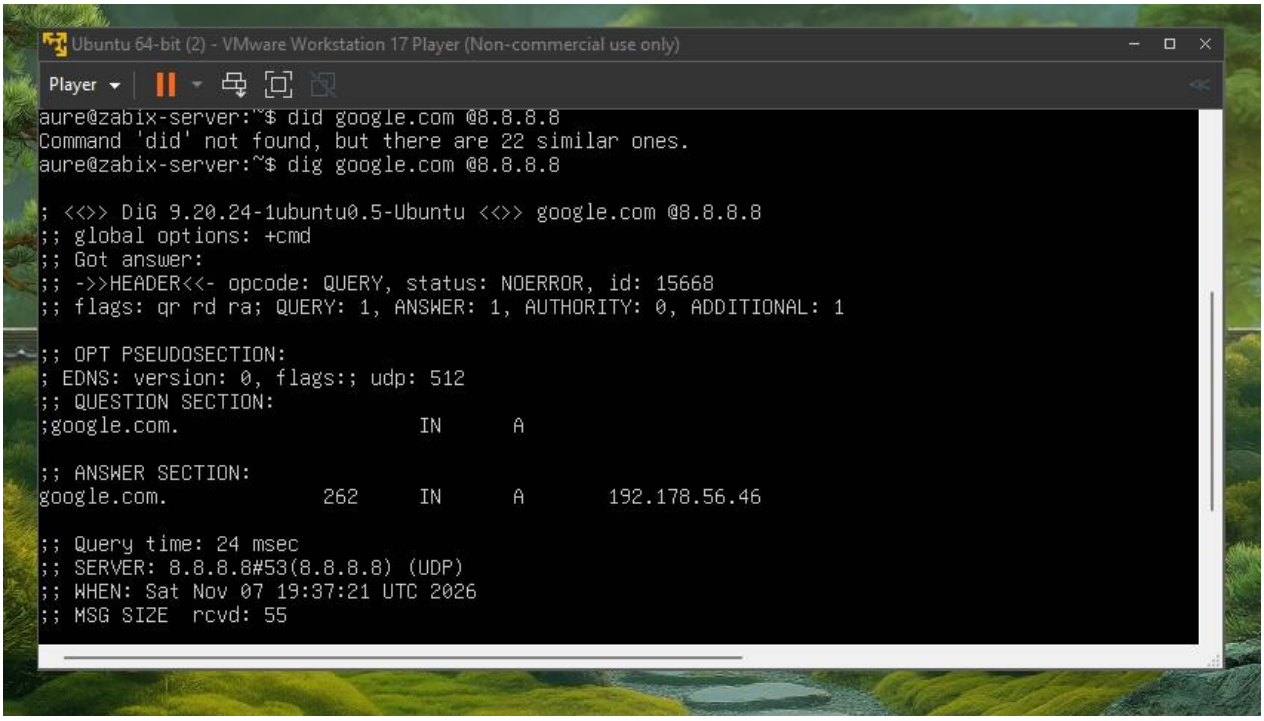
**Práctica 3. Captura de tráfico con tcpdump**

En esta práctica utilizare una máquina virtual con Ubuntu server y lo primero que haremos va ser identificar la interfaz de red disponible con el siguiente comando **ip a**

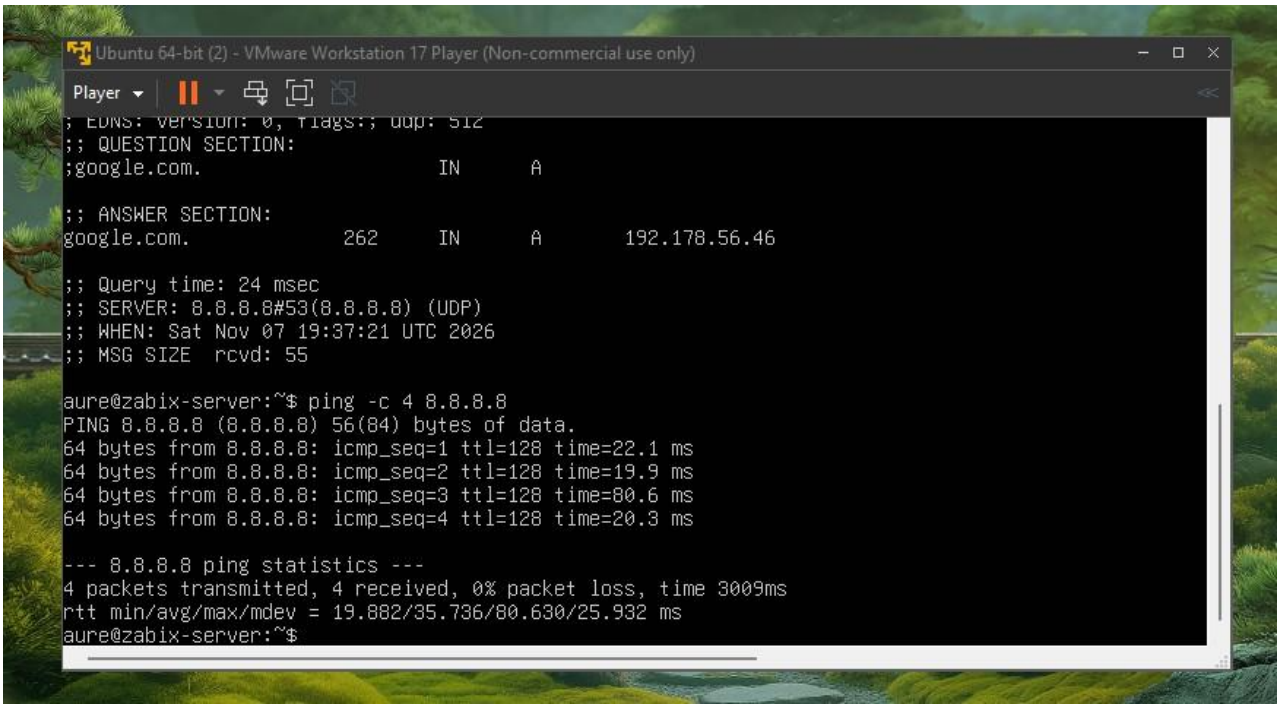
En la imagen vemos que la interfaz de red es ens33 entonces ahora haremos la captura por medio de este comando **sudo tcpdump -i ens33 -w practica3_tcpdump.pcap** y se guarde como pcap



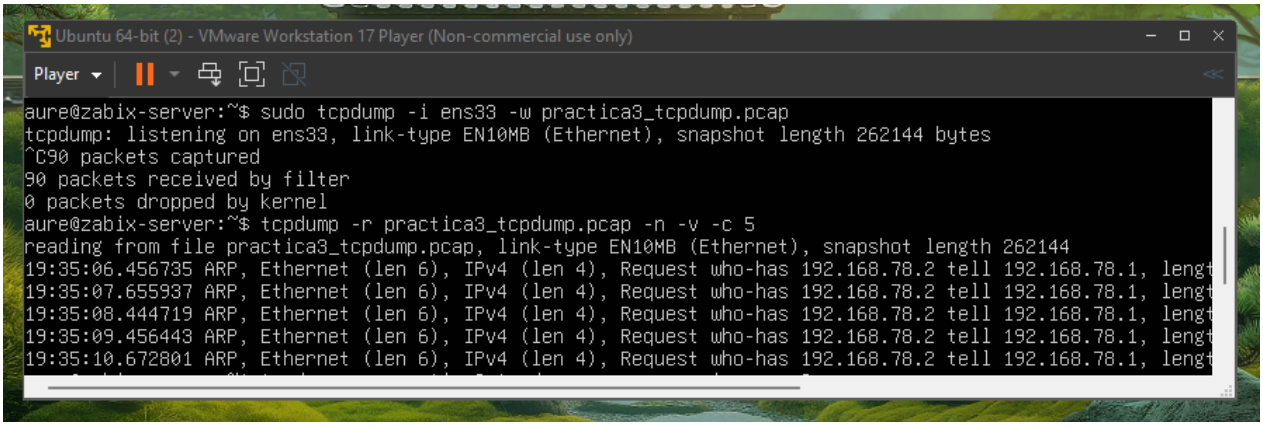

Ahora que está corriendo abri otra terminal donde creare tráfico en la red haciendo una consulta DNS con este comando **dig google.com @8.8.8.8**



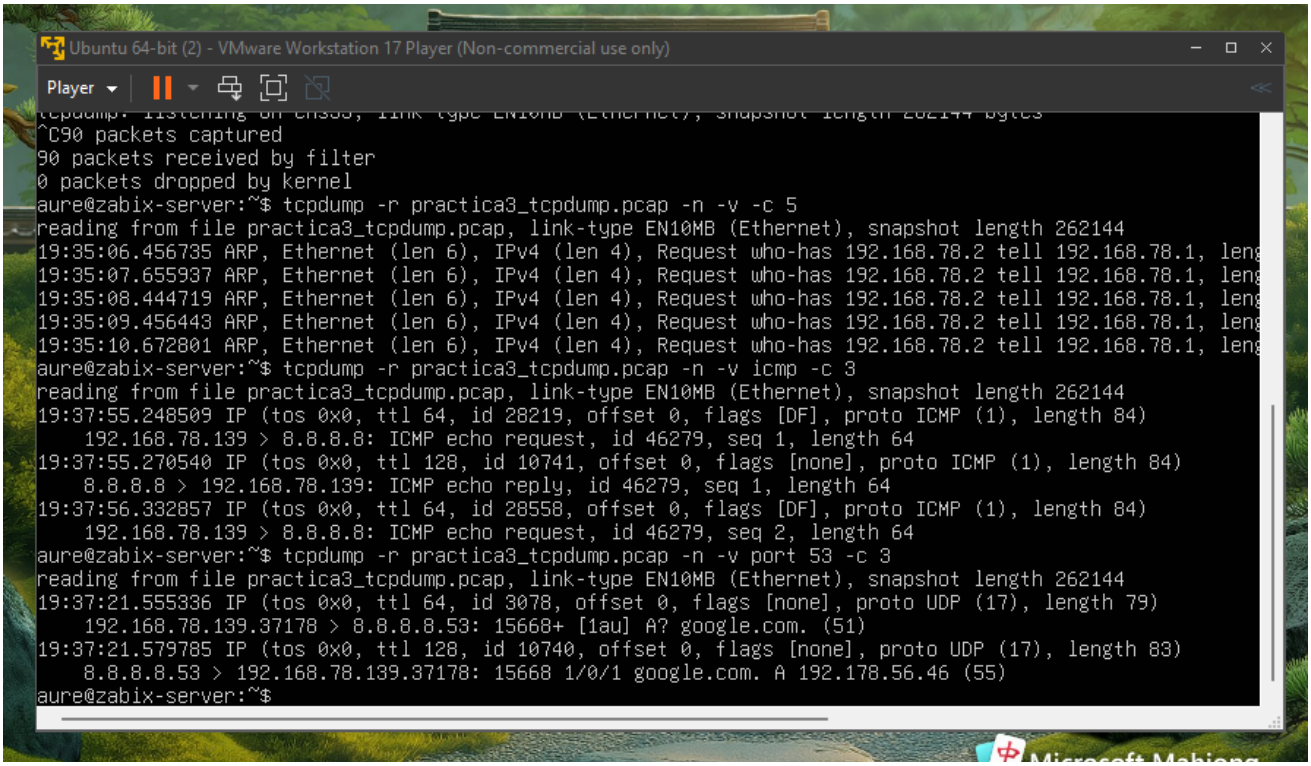
Y otro para generar tráfico ICMP con un ping **ping -c 4 8.8.8.8**




Después vuelvo a la terminal anterior y detengo la captura con Control + C y se vera el resumen y los paquetes capturados pero pondremos el comando **tcpdump -r practica3_tcpdump.pcap -n -v -c 5** para ver los paquetes



Como vimos en la captura anterior no se ven los paquetes que pusimos de prueba los de DNS y ICMP pero para verlos vamos a filtrar entre los paquetes con el comando **tcpdump -r practica3_tcpdump.pcap -n -v icmp -c 3** para ver el tráfico ICMP y el comando **tcpdump -r practica3_tcpdump.pcap -n -v port 53 -c 3** para la consulta DNS como en la siguiente imagen

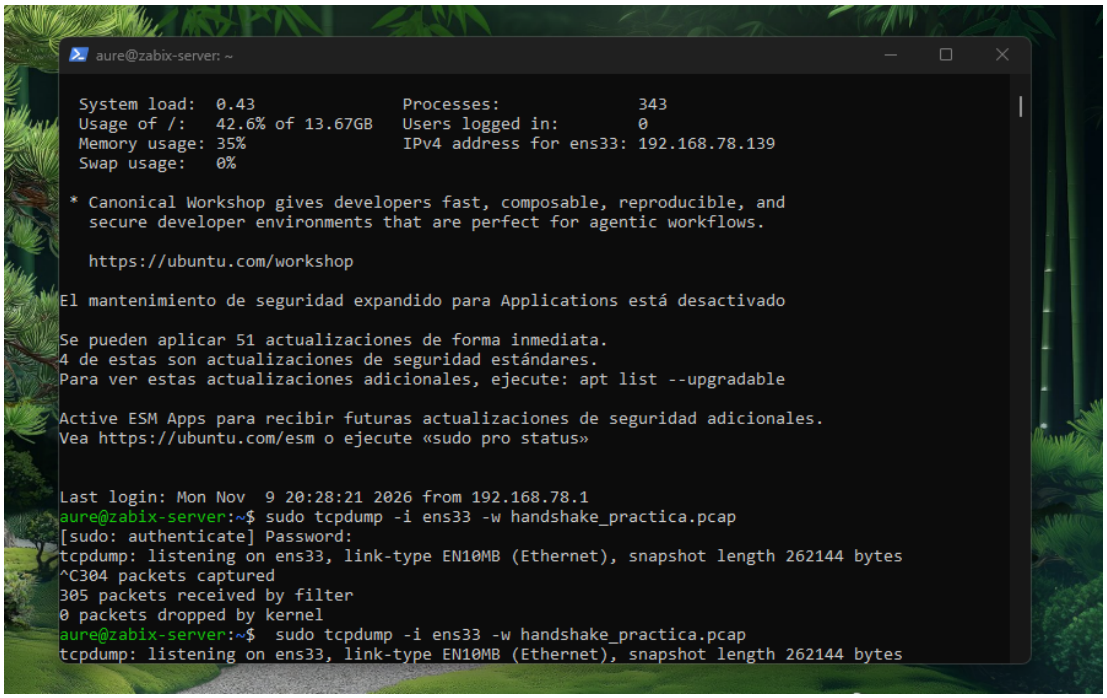



En el protocolo ICMP vemos como hice una prueba de conectividad donde esta ip es el origen 192.168.78.139 y intenta conectarse a esta ip de servidor de prueba de Google 8.8.8.8, vemos que nos da la marca de tiempo de 19:37:55.248509 y nos muestra la solicitud: ICMP echo request, id 46279, seq 1, length 64 que es la solicitud de nuestra máquina virtual Ubuntu hacia el DNS de público de Google. Luego vemos la respuesta de origen .8.8.8.8 hacia mi máquina virtual 192.168.78.139 es como si platicaran y reciben los permisos para comunicarse

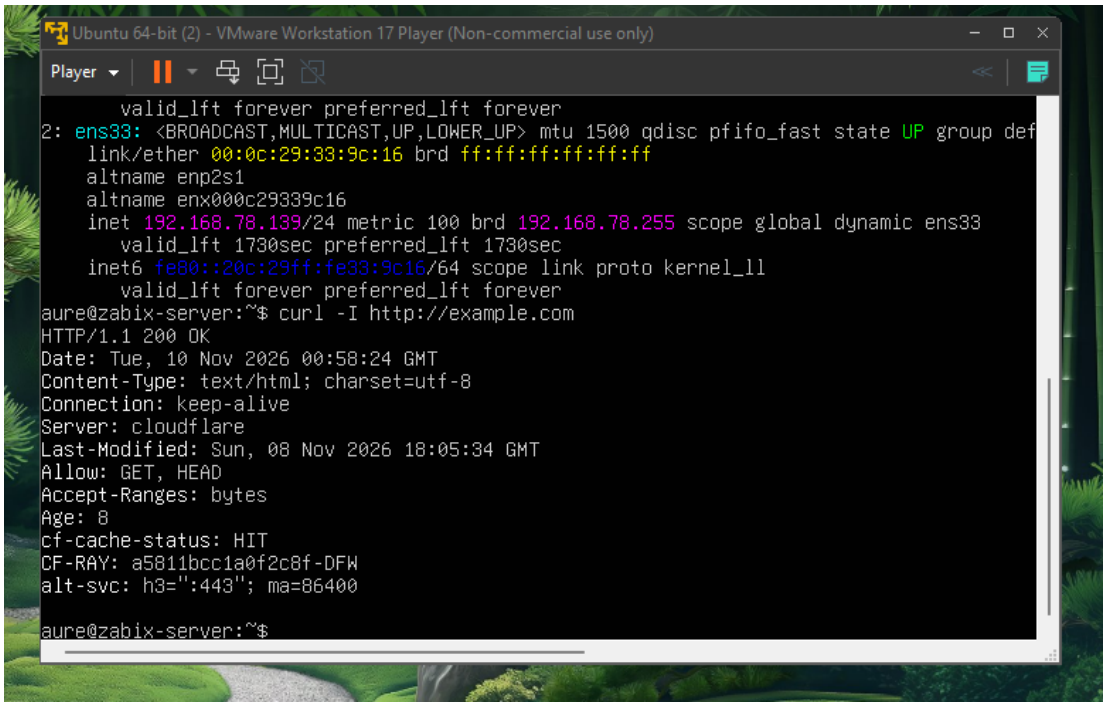
En cuanto el protocolo DNS hice una consulta para ver la ip en formato ipv4 de Google.com, desde el origen 192.168.78.139.37178 hacia 8.8.8.8.53 que es el servidor DNS y luego el servidor le devuelve la respuesta que es ip 192.178.56.46 para el dominio Google.com

**Practica 4. transferir paquetes de Ubuntu Server a Windows**

Primero capturaremos tráfico en Ubuntu con tcpdump guardando el archivo en formato pcap para luego analizarlo con wireshark en Windows le pondré el nombre de handshake_practica.pcap

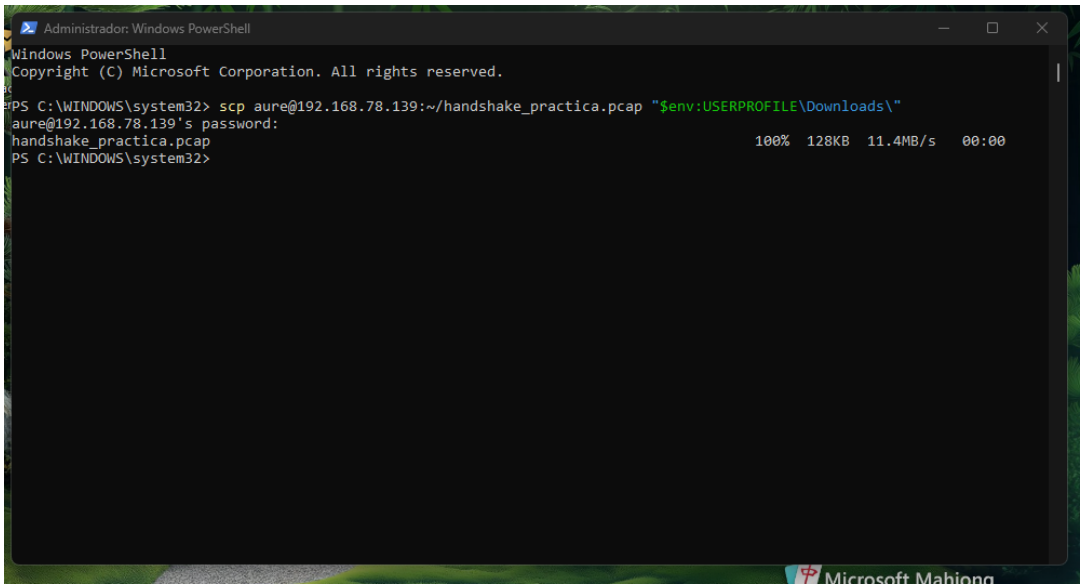



Después abriremos otra terminal en este caso lo voy a poner directo en la terminal de Ubuntu un comando para generar una conexión TCP limpia hacia un servidor web

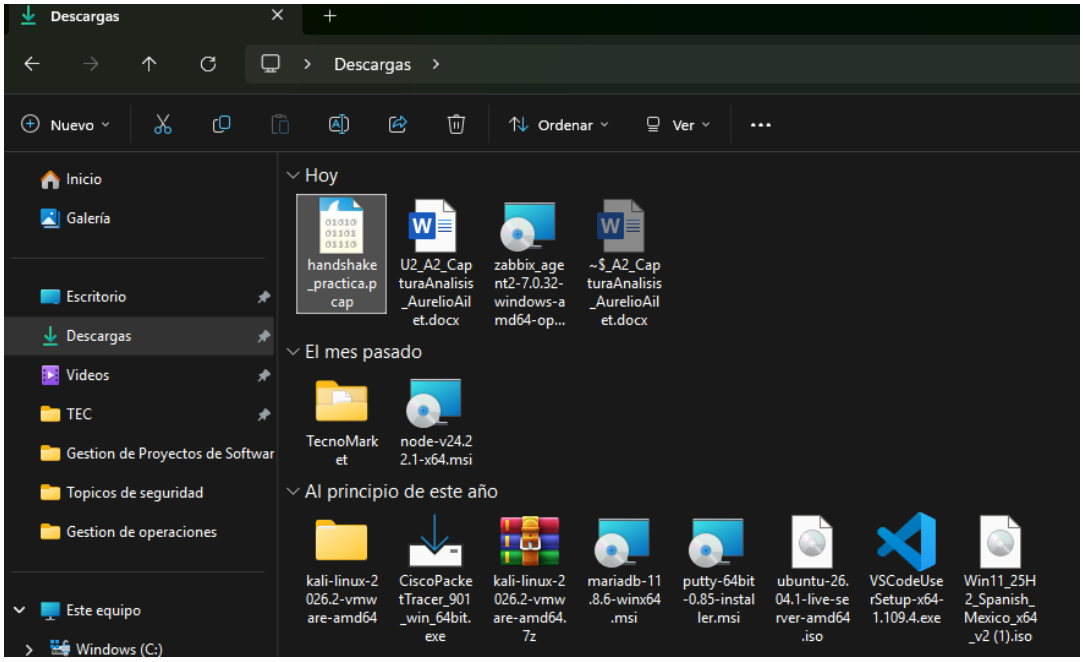


Después volviendo a la otra terminal de Ubuntu detenemos la captura de paquetes con Control+C y asi se guarde la captura

Después pasamos el archivo que capturamos a Windows por medio de PowerShell en Windows de la siguiente manera como se muestra en la captura siguiente, viendo como muestra que se transfirió correctamente marcado el progreso del 100% y donde la guarde en la carpeta de descargas



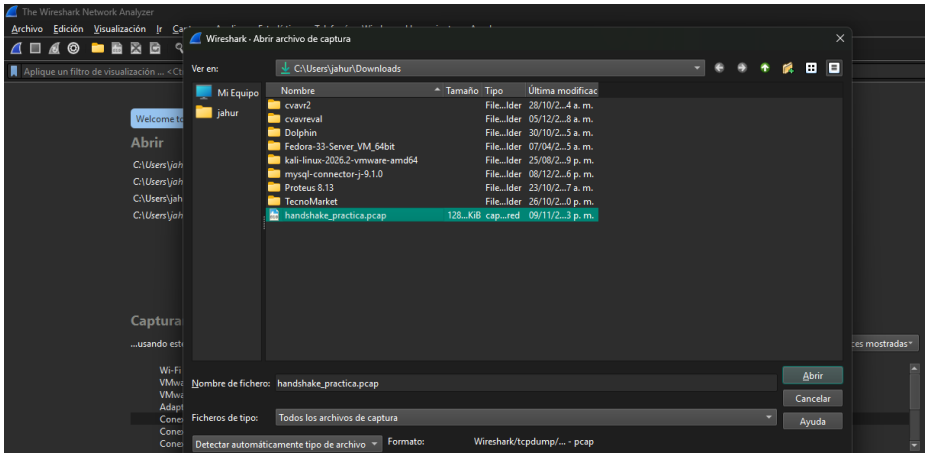

Nos vamos a la carpeta de descargas y vemos que ya se encuentra ahí el archivo



**Practica 5. Analizar el Three-Way Handshake en Wireshark (Windows 11)**

Usando el archi anterior que tranferimos a Windows ahora lo analizaremos para ver el Analizar el Three-Way Handshake en Wireshark

Primero abri el paquete en wireshark de mi Windows




Aquí ya podemos ver los protocolos usé un filtro para solo ver TCP y también le puse en seguir el flujo TCP

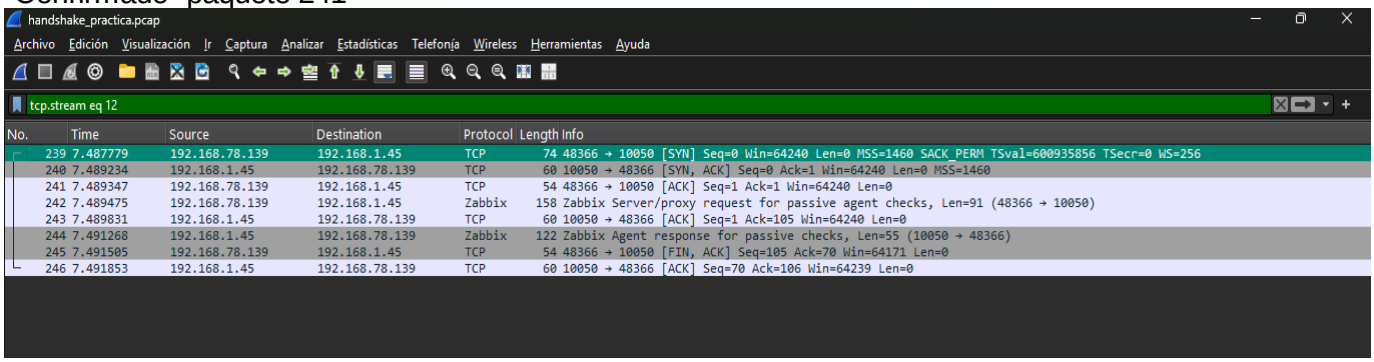
En esta captura podemos ver como se establece la conversación de la siguiente manera:

Mi Laptop SERVIDOR
192.168.1.43 192.168.78.139

———— SYN ————————————►
"Quiero iniciar una conexión" paquete 239

◄———— SYN + ACK ————————————
"Recibido, acepto" paquete 240

———— ACK ————————————►
"Confirmado" paquete 241




También vemos en los siguientes paquetes del 242 en adelante donde se piden datos y en el 245 y 246 se finaliza la conversación con la bandera FIN

En el paquete 239, mi Windows (192.168.1.45) inicia la conexión hacia el servidor (192.168.78.139:10050) en el desglose de cabeceras del protocolo TCP, se observa la bandera Syn: Set (1) activa y el número de secuencia inicial fijado en $Seq = 0$, solicitando la apertura del canal de comunicación.

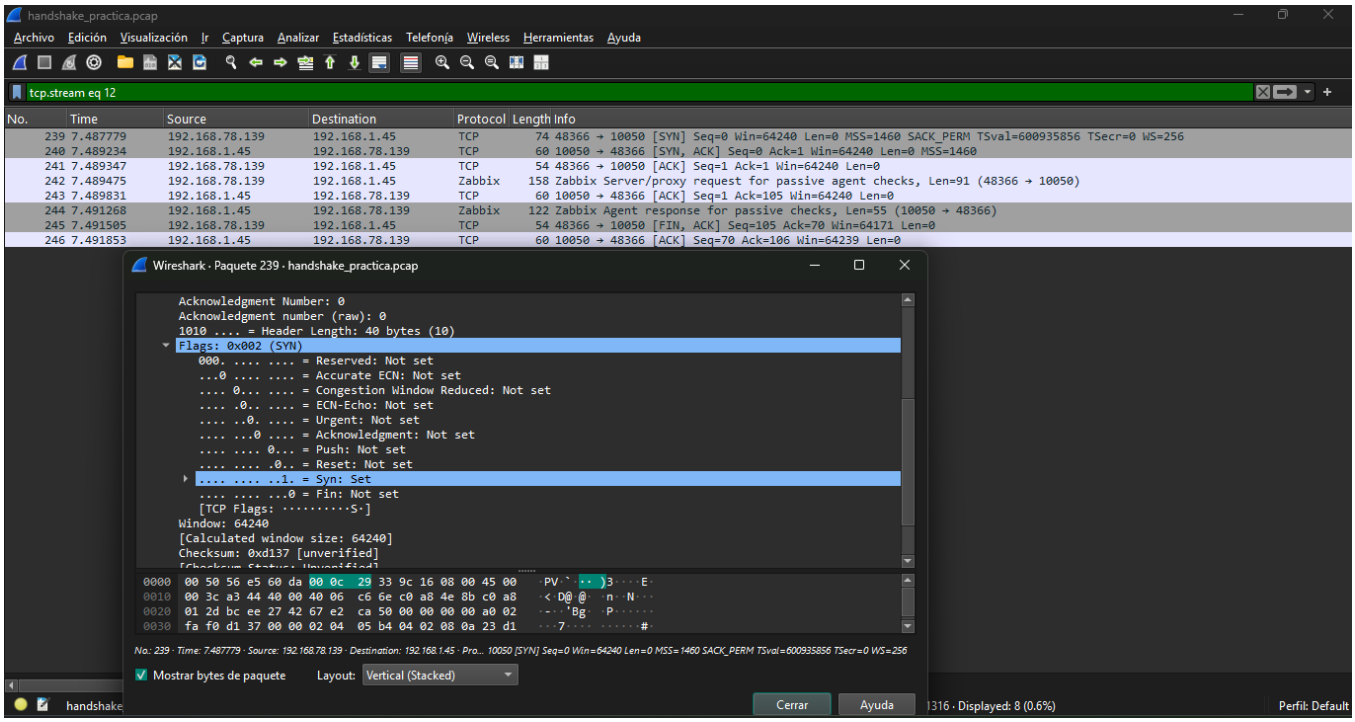


**Tabla Comparativa de Herramientas de Captura de Tráfico**

| Criterio | Wireshark (Práctica 1) | Microsoft Pktmon (Práctica 2) | tcpdump (Práctica 3) |
|---|---|---|---|
| **Software utilizado** | Wireshark (v4.x) | Microsoft Pktmon (Packet Monitor) | tcpdump |
| **Facilidad de instalación y utilización** | Muy alta. Se instala mediante asistente gráfico. Uso intuitivo por su entorno gráfico. | Muy alta. Preinstalado en el kernel de Windows 10/11. Requiere consola como Administrador. | Alta. Preinstalado en distros Linux o mediante sudo apt install tcpdump. Requiere sintaxis CLI. |
| **Tipo de interfaz** | Gráfica (GUI) multilenguaje con panel de paquetes, árbol de protocolos y visor hexadecimal. | Línea de comandos (CLI) en CMD o PowerShell de Windows. | Línea de comandos (CLI) nativa de terminal Linux/Unix. |
| **Información que permite visualizar** | Desglose de capas OSI (2 a 7), árbol de protocolos, payload y flujos (Follow TCP/TLS Stream). | Metadatos ETW, ID de componentes de red, longitud y encabezados. (Requiere conversión a .pcapng). | Timestamps, IP/MAC, puertos, banderas TCP, longitud, secuencia/ACK y ASCII/Hex (-X). |
| **Protocolos identificados** | DNS, ICMP, HTTP, HTTPS (TLS 1.2/1.3), QUIC, TCP, UDP, ARP, entre otros. | ICMP, DNS, TCP, UDP y eventos a nivel de kernel/driver de Windows. | ICMP, DNS, ARP, TCP, UDP, SSH, HTTP y protocolos sobre IP/Ethernet. |
| **Facilidad para filtrar paquetes** | Excelente. Filtros en tiempo real con sintaxis expresiva (ej. ip.addr == X, dns). | Media. Filtros previos con pktmon filter add por IP, puerto (-p) o protocolo (-t). | Excelente en CLI. Utiliza lenguaje BPF en captura o lectura (ej. tcpdump -r file.pcap icmp). |

| | | | |
|---|---|---|---|
| **Ventajas** | • Interfaz visual potente y análisis forense.<br>• Reconstrucción de flujos.<br>• Coloreado por tipo. | • 0% impacto (sin Npcap).<br>• Diagnóstico nativo de Windows.<br>• Detecta paquetes drop a nivel kernel. | • Mínimo consumo de recursos.<br>• Ideal para servidores sin GUI (Headless).<br>• Automatizable con scripts Bash. |
| **Limitaciones** | • Alto consumo de RAM.<br>• Requiere entorno gráfico.<br>• Requiere controladores (Npcap). | • Sintaxis rígida y comandos extensos.<br>• Sin análisis de capas superiores nativo.<br>• Obliga a convertir .etl a .pcapng. | • Curva de aprendizaje CLI.<br>• Complejo para inspeccionar flujos pesados o binarios sin GUI. |
| **Diferencias encontradas en las capturas** | Muestra el paquete una sola vez por la interfaz física, facilitando la lectura secuencial. | Registra múltiples repeticiones de un mismo paquete por el rastreo de eventos (ETW) en el kernel | Muestra una vista limpia por paquete en la interfaz en formato de texto continuo. |

**Preguntas de reflexión**

**1. ¿Qué diferencias encontraron entre las tres herramientas?**

Pues wireshark la Interfaz gráficaes másfácilde entender viene coloreado por protocolos y captura única por interfaz física. Microsoft Pktmon es nativa de Windows sin dependencias de terceros, registra eventos internos a nivel de kernel generando duplicidad del mismo paquete en las capas del SO. Tcpdump es una herramienta ligera en Linux, para servidores sin entorno gráfico y automatización con scripts Bash.

**2. ¿Cuál permite visualizar con mayor facilidad los detalles de una trama?**

**Wireshark s**u entorno gráfico permite inspeccionar encabezados, bytes en hexadecimal, texto claro y flujos completos con un solo clic.

**3. ¿Qué información puede obtenerse al analizar los paquetes de una comunicación?**

- Direcciones IP de origen/destino y MAC address.
- Puertos y protocolos utilizados
- Marcas de tiempo, latencia y banderas de control
- Nombres de dominio consultados y datos en texto plano

**4. ¿Por qué es importante identificar los protocolos que circulan en una red?**

Permite verificar qué servicios están activos, asegurar que la comunicación cumpla los estándares de la organización, optimizar el uso de ancho de banda y detectar protocolos no autorizados o inseguros

**5. ¿Cómo puede ayudar la captura de tráfico a detectar problemas o incidentes de seguridad?**

Diagnóstico de red: Identifica latencias elevadas, pérdida de paquetes o fallas en resolución DNS. Seguridad: Detecta escaneos de puertos, exfiltración de información, ataques de denegación de servicio, tráfico malicioso dirigiéndose a IPs no confiables.

**6. ¿Cuál de las tres herramientas recomendarían para realizar un diagnóstico básico de red? Justifiquen su respuesta.**

En mi caso Wireshark es la más recomendada porque su interfaz visual y su sistema de filtros en tiempo real facilitan la interpretación inmediata de los eventos sin necesidad de memorizar comandos o convertir formatos de archivo.

**Conclusión**

En conclusión en estas tres practicas comparamos como funcionan estos tres herramientas para analizar paquetes de protocolos como por ejemplo consultas DNS y trafico ICMP, donde pudimos ver que wireshark es la mas practica porque tiene una interfaz mas intuitiva donde podemos ver a detalle y muy profundamente la información de los protocolos y como se comunican, o conversan entre ellos, en cambio las demás son en línea de comandos y puede ser más difícil porque tienes que conocer los comandos de las funciones, pero la ventaja es que ya vienen preinstalados en sus respectivos sistemas operativos, Microsoft Pktmon por ejemplo una ventaja es que la mayoría tenemos Windows asi que es muy fácil acceder pero no da detalles de los paquetes solo captura y los guarda, en cambio Tcpdump de Linux puedes capturar y guardar, además te deja ver los detalles de los paquetes y filtrar entre ellos.